

Progetto di Reti Logiche (Prof Fabio Salice)

Samuele Faccincani: matricola 959536, codice persona 10744304

Anno accademico 2024-2025

Contents

1	Introduzione	2
1.1	Scopo del progetto	2
1.2	Specifica	2
1.3	Interfaccia dei componenti	2
2	Design	4
2.1	Circuito	5
2.2	Macchina a stati	6
3	Scelte progettuali	8
3.1	Componenti	8
4	Risultati	10
5	Conclusione	10

1 Introduzione

1.1 Scopo del progetto

Lo scopo del progetto è quello di leggere una sequenza di K parole W applicando un filtro differenziale, che può essere di ordine 3 o di ordine 5, e salvare la sequenza K di nuove parole R appena dopo la prima. La funzione matematica che descrive il filtro è $f(i) = (1/n) * \sum C_j * f[j+i]$ con j che varia da -l a +l. Per il filtro di ordine 3, l sarà 2 e n 12, mentre per il filtro di ordine 5 i valori saranno rispettivamente 3 e 60. I valori prima e dopo sequenza sono da considerare nulli nel calcolo del filtro. La sequenza filtrata verrà scritta alla fine dei valori da elaborare.

1.2 Specifica

Il modulo ha 2 ingressi primari, START (1 bit), ADD (16 bit) e una uscita primaria DONE (1 bit). Sono presenti altri due segnali, RESET (asincrono) e CLK, segnale di clock unico per tutto il sistema.

1.3 Interfaccia dei componenti

entity project_reti_logiche is

```
port (
    i_clk : IN STD_LOGIC;
    i_rst : IN STD_LOGIC;
    i_start : IN STD_LOGIC;
    i_add : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
    o_done : OUT STD_LOGIC;
    i_mem_data : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
    o_mem_addr : OUT STD_LOGIC_VECTOR(15 DOWNTO 0);
    o_mem_data : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);
    o_mem_we : OUT STD_LOGIC;
    o_mem_en : OUT STD_LOGIC
);
end project_reti_logiche;
```

in particolare

- i_clk e i_rst rappresentano il clock e il reset
- i_start è il segnale di start
- i_add rappresenta l'indirizzo da cui parte la sequenza
- o_done rappresenta la fine dell'elaborazione
- o_mem_addr rappresenta l'indirizzo mandato alla memoria

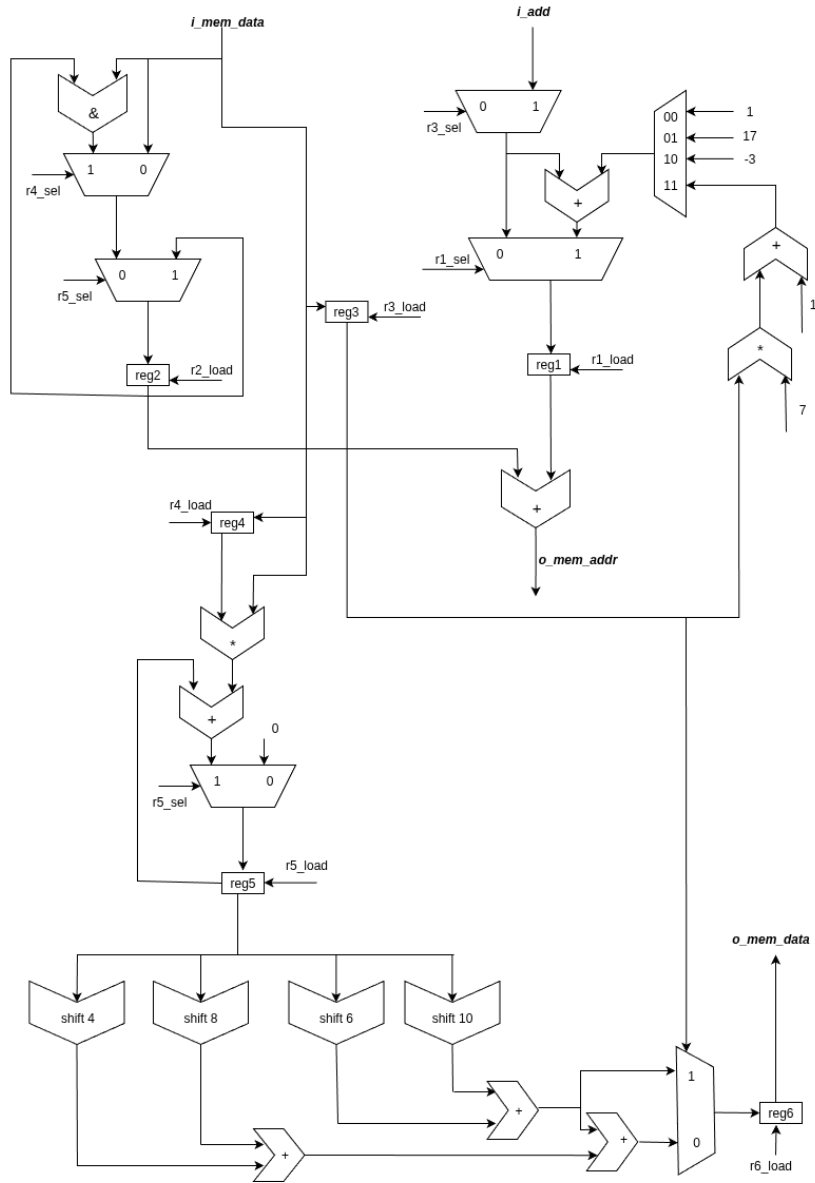
- `i_mem_data` rappresenta il dato che arriva dalla memoria a seguito di una richiesta di lettura
- `o_mem_data` rappresenta il valore che verrà scritto in memoria
- `o_mem_en` rappresenta il segnale di `ENABLE` per comunicare con la memoria (sia lettura che scrittura)
- `o_mem_we` rappresenta il segnale di scrittura, se `=1` il modulo può scrivere, se `=0` il modulo può leggere

2 Design

La prima parte del progetto è stata spesa nella creazione di un circuito per processare i segnali in input e produrre i corretti segnali in output. Il circuito viene comandato da una macchina a stati.

Di seguito sono riportati il circuito e la macchina a stati utilizzati per scrivere il codice vhd.

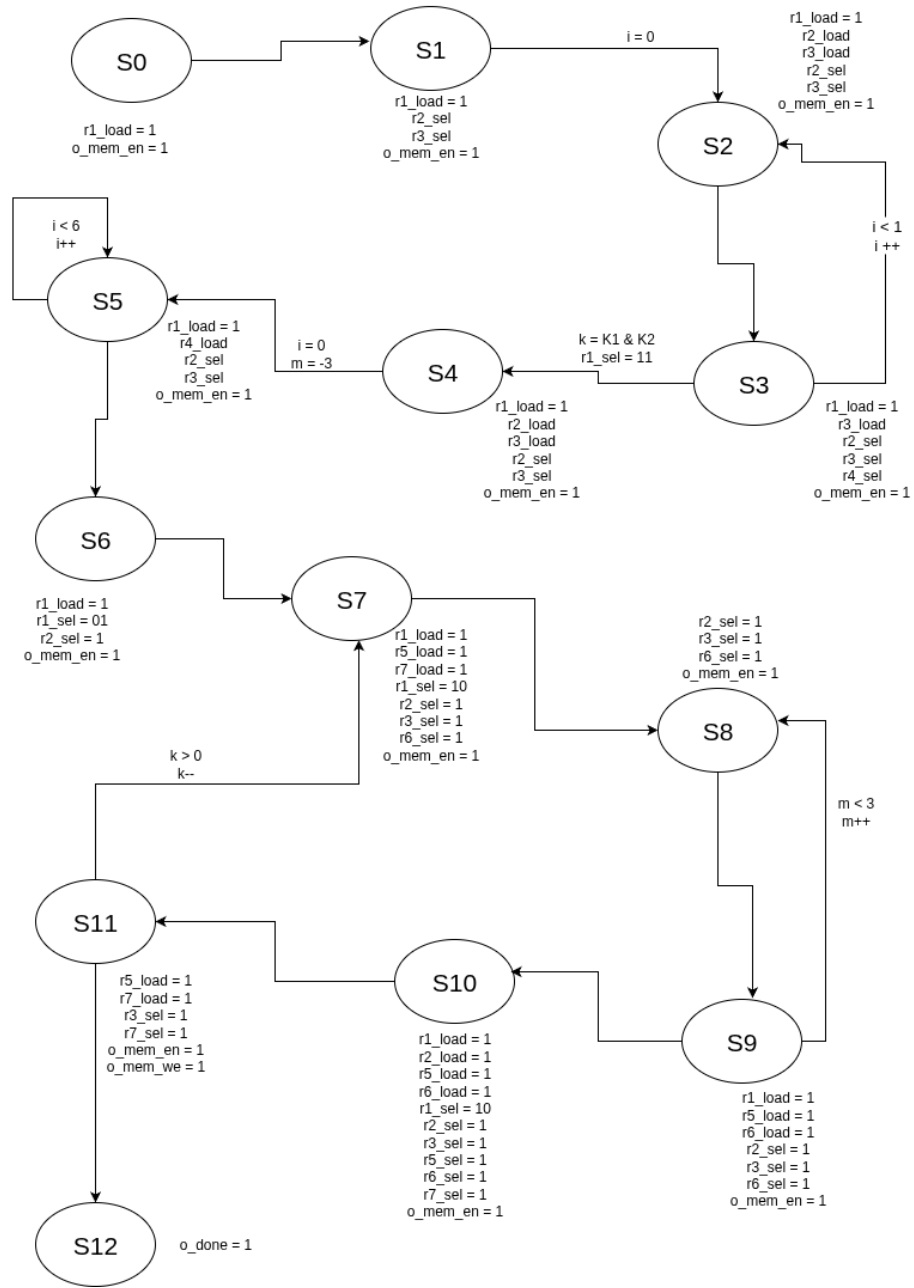
2.1 Circuito



2.2 Macchina a stati

Stati della FSM

- S0: Stato iniziale in cui si attende $i_start=1$
- S1: Dopo $start=1$, leggo il contenuto di i_add
- S2: Primo stato del ciclo, leggo il Ki , concateno con il segnale già presente nel registro
- S3: Incremento l'indirizzo di memoria per leggere il dato successivo
- S4: Leggo il dato che indica il filtro da utilizzare
- S5: Salvo la sequenza Ci
- S6: Accedo alla prima parola della sequenza
- S7: Accedo al primo valore che utilizzerò nel filtro
- S8: Leggo il valore
- S9: Aggiorno la parola normalizzata, accedo al prossimo valore
- S10: Salvo il valore finale, aggiorno il numero di parole mancanti e accedo alla posizione di scrittura
- S11: Scrivo in memoria e accedo alla parola successiva da normalizzare
- S12: stato in cui termina la computazione



3 Scelte progettuali

3.1 Componenti

Per avere una gestione migliore dei segnali, oltre al componente principale mi sono avvalso di una seconda entity (chiamata "datapath") che mi ha permesso di dividere la logica del circuito dalla logica della macchina a stati, per avere un codice più comprensibile sia in fase di lettura che in fase di debug.

La specifica di questa entity è le seguente:

```
entity datapath is
  Port (
    i_clk : in std_logic;
    i_rst : in std_logic;
    i_mem_data : in std_logic_vector (7 downto 0);
    i_addr : in std_logic_vector (15 downto 0);
    o_mem_data : out std_logic_vector (7 downto 0);
    o_mem_addr : out std_logic_vector (15 downto 0);
    r1_load : in std_logic;
    r2_load : in std_logic;
    r3_load : in std_logic;
    r4_load : in std_logic;
    r5_load : in std_logic;
    r6_load : in std_logic;
    r1_sel : in std_logic_vector(1 downto 0);
    r2_sel : in std_logic;
    r3_sel : in std_logic;
    r4_sel : in std_logic;
    r5_sel : in std_logic;
    r6_sel : in std_logic;
    r7_sel : in std_logic;
    j : out std_logic_vector(15 downto 0);
    i : in integer
    m : in integer
  );
end datapath;
```


I segnali non presenti nella specifica vista precedentemente sono:

- r1_load, r2_load, r3_load, r4_load, r5_load, r6_load servono per abilitare il salvataggio di valori nei rispettivi registri
- r2_sel, r3_sel, r4_sel, r5_sel, r6_sel, r7_sel sono utilizzati per indirizzare i rispettivi multiplexer
- j, i, m sono utilizzati per gestire rispettivamente il numero delle parole rimanenti, il coefficiente da utilizzare e il numero di parole da utilizzare per ottenere la parola finale.

4 Risultati

Per verificare la correttezza del codice mi sono affidato in primis al test bench "base" per verificare la correttezza a grandi linee del codice, mentre per controllare l'esattezza dei calcoli e del funzionamento dei registri ho utilizzato un testbench singolo ma che conteneva vari scenari: il primo e il secondo controllavano la validità dei calcoli sia del filtro di ordine 5 che quello di ordine 3 con circa 12000 parole, il terzo —, il quarto —, il quinto verificava che il dimensionamento dei vettori intermedi fosse opportuno per gestire tutti i numeri possibili, il sesto

Riporto qui i risultati sia del report_timing che del report_utilization, in cui possiamo rispettivamente vedere come sia stato rispettato il limite di tempo richiesto e non siano presenti latch utilizzati come memoria.

Timing Report

Slack (MET) : 10.307ns (required time - arrival time)

1. Slice Logic	Site Type	Used	Fixed	Available	Util %
	Slice LUTs*	498	0	134600	0.37
	LUT as Logic	498	0	134600	0.37
	LUT as Memory	0	0	134600	0.00
	Slice Registers	208	0	269200	0.08
	Register as Flip Flop	208	0	269200	0.08
	Register as Latch	0	0	269200	0.00
	F7 Muxes	0	0	67300	0.00
	F8 Muxes	0	0	33650	0.00

5 Conclusione

Data la varietà di scenari analizzati e il superamento rispettando i vincoli temporali e non inferendo nessun latch sia nella simulazione "Behavioural" che nella "Post-Synthesis Functional" posso affermare che il progetto è stato completato rispettando le richieste della specifica.